// EVOLUINDO NA PROGRAMAÇÃO

Python Intermediário

Estruturas de dados avançadas, coleções, laços de repetição e uso de bibliotecas.

Listas: Agrupando Múltiplos Dados

Listas são coleções ordenadas e **mutáveis**, delimitadas por colchetes `[]`. Podem armazenar qualquer tipo de dado.

Indexação: A contagem dos elementos inicia sempre no índice `0`.

Heterogêneas: Aceitam textos, números e booleanos na mesma estrutura.

```
# Criando uma lista de frutas
frutas = ["maçã", "banana", "uva"]

# Acessando o primeiro elemento
print(frutas[0]) # Saída: maçã
```

Manuseio e Manipulação de Listas

Adicionar e Inserir

Insere elementos no final ou em posições específicas.

```
lista.append("laranja")
lista.insert(0, "pera")
```

Remover Elementos

Exclui itens por valor ou posição exata.

```
lista.remove("banana")
lista.pop(1)
```

Ordenar e Contar

Organiza a ordem e verifica o tamanho total.

```
lista.sort()
tamanho = len(lista)
```

Tuplas: Dados Imutáveis

Tuplas são semelhantes às listas, porém **imutáveis** (não podem ser alteradas após criadas). Utilizam parênteses `()`.

Segurança: Ideais para dados fixos que não devem sofrer modificações acidentais.

Performance: Ocupam menos memória e processamento que listas.

```
# Tupla de coordenadas fixas
coordenadas = (15.5, 48.2)

# Tentativa de alteração gera erro:
# coordenadas[0] = 10.0 (TypeError)
print(coordenadas[0])
```

O Laço de Repetição for

O comando `for` percorre iterativamente cada item de uma sequência (como listas, tuplas ou textos).

Perfeito para automatizar tarefas repetitivas sem precisar reescrever código.

```
notas = [8.0, 9.5, 6.0]

for nota in notas:
    print("Nota registrada:", nota)
```

Gerando Sequências com range()

A função `range()` gera intervalos numéricos automaticamente, sendo amplamente usada em loops `for`.

range(5): De 0 até 4.

range(1, 6): De 1 até 5.

range(0, 10, 2): De 0 a 8 pulando de 2 em 2.

```
# Contando de 1 até 5
for i in range(1, 6):
    print(i)
```

O Laço de Repetição while

O loop `while` executa um bloco repetidamente **enquanto** uma condição lógica for verdadeira.

Cuidado: é essencial atualizar a variável de controle para evitar loops infinitos!

```
tentativas = 0

while tentativas < 3:
    print("Tentativa número:", tentativas)
    tentativas += 1
```

Bibliotecas: Superpoderes Prontos

O que são?

Módulos de código criados por terceiros para resolver problemas complexos rapidamente.

Como importar?

Utilizamos a palavra-chave `import` para trazer ferramentas externas para o seu script.

Ecossistema

Milhares de pacotes disponíveis para estatística, gráficos, web e IA.

Bibliotecas Nativas Úteis

O Python já vem com diversas bibliotecas nativas prontas para uso imediato sem instalações complexas.

math: Funções matemáticas avançadas.

random: Geração de números aleatórios.

datetime: Manipulação de datas e horários.

```
import random

# Sorteando um número entre 1 e 10
sorteado = random.randint(1, 10)
print("Número sorteado:", sorteado)
```

Integrando os Conceitos

Combinando listas, laços de repetição e condicionais criamos sistemas inteligentes e dinâmicos.

Exemplo: Filtrar elementos ou verificar presenças em grandes conjuntos de dados.

```
idades = [14, 22, 17, 19]

for idade in idades:
    if idade ⩾ 18:
        print(idade, "→ Maior de idade")
    else:
        print(idade, "→ Menor de idade")
```

Resumo da Etapa Intermediária

Coleções

Listas mutáveis e tuplas imutáveis para organizar dados.

Repetições

Loops `for` e `while` para fluxos contínuos e automáticos.

Bibliotecas

Módulos externos e nativos para ampliar suas capacidades.

Parabéns pelo Avanço!

Você agora domina estruturas de dados essenciais e controle de fluxo avançado em Python.

```
import sucesso; sucesso.iniciar()
```